Manual de Instalação Verificador de Preços

Projeto: TCC — Engenharia de Software

Ambiente: Windows + cellular Android

Repositório: https://github.com/marlomlucas/tcc

Referência: novembro de 2026

Identidade visual: ATL Software

Objetivo: instalar o backend Django e o MySQL em um computador novo, restaurar produtos de demonstração, iniciar a API pela rede local e consultar preços no aplicativo Android.

1. Requisitos do computador e do celular

Item	Requisito
Sistema operacional	Windows 10/11 (64 bits), para usar o inicializador .bat.
Python	3.14.x, conforme o ambiente de desenvolvimento.
Banco de dados	MySQL Community Server 8.0; MySQL Workbench recomendado.
Download do projeto	Git para Windows ou ZIP do GitHub.
Celular	Android com câmera (mínimo Android 6); validado no Moto G22, Android 12.
Rede	Computador e celular conectados à mesma rede, sem isolamento entre dispositivos.

Downloads oficiais: Python: https://www.python.org/downloads/windows/

MySQL: https://dev.mysql.com/downloads/installer/

Git: https://git-scm.com/downloads/win

O aplicativo Android será distribuído como **“varificador_precos.apk”**, na **raiz do projeto**

Exemplo: (D:\Tcc\varificador_precos.apk). código-fonte continua disponível na pasta android-verificador-precos.

2. Obter o projeto pelo GitHub

No **PowerShell**, escolha uma pasta para os arquivos. Os comandos abaixo utilizam **D:\Tcc** como exemplo (substitua a unidade se necessário):

```
cd D:\
git clone https://github.com/marlomlucas/tcc.git Tcc
cd D:\Tcc\backend-verificador-precos
```

3. Instalar MySQL e preparar o banco de dados

Instale **MySQL Server 8.0** e, preferencialmente, o **MySQL Workbench**. Defina a senha do administrador MySQL durante a instalação e mantenha a porta padrão **3306**. Certifique-se de que o serviço MySQL esteja ativo.

Nesta instalação, crie um banco vazio. O Django criará as tabelas na etapa 6 por meio de migrate. Para os produtos, utilize somente backup_banco\produtos_demonstracao.sql após as migrations. Não restaure o backup completo nesta etapa.

Criar o banco vazio

Abra o MySQL Workbench, conecte-se ao servidor como administrador e execute:

```
CREATE DATABASE IF NOT EXISTS verificador_precos
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

O banco verificador_precos precisa existir antes do migrate. Não crie tabelas manualmente: as migrations do Django definem a estrutura e registram o histórico de alterações.

Para confirmar a criação do banco, execute no Workbench:

```
SHOW DATABASES LIKE 'verificador_precos';
```

Criar o usuário MySQL da aplicação

No Workbench, conectado como administrador MySQL, crie um usuário para a aplicação com uma senha definida pelo avaliador:

```
CREATE USER 'verificador_app'@'localhost'
  IDENTIFIED BY 'SENHA_ESCOLHIDA_PARA_O_APLICATIVO';
GRANT ALL PRIVILEGES ON verificador_precos.*
  TO 'verificador_app'@'localhost';
```

Guarde o usuário verificador_app e sua senha: os mesmos valores serão utilizados no .env. Se o usuário já existir, não execute CREATE USER novamente.

4. Criar o ambiente Python e instalar as dependências

No **PowerShell**, dentro da pasta do projeto “D:\Tcc\backend-verificador-precos”

```
cd D:\Tcc\backend-verificador-precos
py -3.14 --version
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Se o comando py não estiver disponível, utilize python após confirmar que ele aponta para Python 3.14.x. **O ambiente .venv deve ser criado no próprio computador; não deve ser copiado do GitHub.**

5. Criar o .env a partir do .env.example

O projeto já fornece o modelo **“backend-verificador-precos\.env.example”**. **Não é necessário criar o “.env” do zero.**

Faça seguinte passo a passo:

1. Abra o **PowerShell** na pasta (Local do Projeto) D:\Tcc\backend-verificador-precos.
2. Copie o arquivo de exemplo com o comando abaixo; isso cria o arquivo .env na mesma pasta.

```
Copy-Item .env.example .env
```

3. Abra o .env no Bloco de Notas e substitua os valores de exemplo pelos dados da instalação.

4. Identifique o endereço IPv4 do computador onde será rodado o servidor. Ele deve estar na mesma rede onde ira se conectar o celular:

   Abra o CMD e execute “ipconfig” procure por IPv4. Ex: 192.168.100.10

5. Preencha as variáveis conforme o exemplo (**o IP “192.168.100.10” é fictício**):

```
DB_NAME=verificador_precos
DB_USER=verificador_app
DB_PASSWORD=SENHA_ESCOLHIDA_PARA_O_APLICATIVO
DB_HOST=localhost
DB_PORT=3306

DJANGO_SECRET_KEY=CHAVE_A_SER_GERADA
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=127.0.0.1,localhost,192.168.100.10

IMAGENS_BASE_URL=http://192.168.100.10:8000/
```

6. Gere uma chave Django no terminal e cole o resultado em DJANGO_SECRET_KEY:

   .\.venv\Scripts\python.exe -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"

7. **Importante para as imagens:**

   Remova a linha **IMAGENS_STORAGE_ROOT=D:/CAMINHO_DO_PROJETO/media** que veio no .**env.example**, ou substitua-a pelo diretório real (ex.: D:/Tcc/backend-verificador-precos/media). Se remover a variável, o Django utilizará automaticamente a pasta media do backend.

8. Salve o arquivo com o nome exato **“.env”** (não .env.txt).

   Atenção: Diferença entre os endereços: o Android descobre o IP da API pelo mDNS. O endereço para baixar imagens é recebido no JSON e é construído pelo Django usando IMAGENS_BASE_URL. Para esta demonstração, API e imagens ficam no mesmo computador. Quando o IPv4 mudar, atualize DJANGO_ALLOWED_HOSTS e IMAGENS_BASE_URL e reinicie o servidor.

6. Executar migrations, importar produtos e criar administrador

Com o MySQL ativo, o banco vazio criado e o .env preenchido, execute no PowerShell, na pasta do backend:

```
.\.venv\Scripts\python.exe manage.py check
.\.venv\Scripts\python.exe manage.py migrate
```

Após concluir migrate, importe os cinco produtos pelo Prompt de Comando (CMD), não pelo PowerShell:

```
cd /d D:\Tcc (ou caminho do projeto)
mysql --default-character-set=utf8mb4 -u verificador_app -p verificador_precos < backup_banco\produtos_demonstracao.sql
```

Confira no Workbench: SELECT COUNT(*) FROM verificador_precos.produtos; (esperado: 5). Se mysql não for reconhecido, abra o SQL no Workbench por File → Open SQL Script. Importe apenas uma vez.

Por fim, volte ao PowerShell na pasta do backend e crie um administrador do Django:

```
.\.venv\Scripts\python.exe manage.py createsuperuser
```

7. Verificar as imagens dos produtos

As imagens são arquivos separados do MySQL. O local esperado é:

D:\Tcc\backend-verificador-precos\media\img_produtos

O arquivo de cada fotografia tem o formato CODIGO_DE_BARRAS.webp. Confira se as imagens de demonstração foram disponibilizadas no GitHub e baixadas para essa pasta.

Para enviar uma foto JPG, PNG ou WebP de até 5 MB, acesse o Django Admin (http://127.0.0.1:8000/admin/). O backend converte a foto para WebP. A URL retornada na API pode incluir ?v= para atualizar o cache.

8. Iniciar o servidor pelo arquivo .bat

Na pasta "**Tcc\backend-verificador-precos"**, contém o arquino **"iniciar-servidor.bat"**. Ele serve para inicializar o servidor. Ao ser executado ira mostrar o menu com as seguintes opções.

- **1 — LOCAL (OFF-LINE):** inicia Django apenas em 127.0.0.1:8000, sem anunciar o mDNS. Útil para testar o Admin e a API no computador.
- **2 — REDE (ON-LINE):** lista interfaces e IPv4; selecione a interface que compartilha a rede do Android (ou pressione **M** para informar o IP). Abre dois terminais: **Django** e **mDNS**.
- **0 — Sair.**

No modo rede, o processo mDNS aguarda a API ficar disponível por aproximadamente 30 segundos e anuncia o serviço **_vprecos._tcp.local.** Confira no terminal a mensagem de anúncio ativo. **On-line** significa rede local; não exige internet, mas o MySQL precisa estar funcionando.

Para encerrar o sistema, use **Ctrl+C no terminal mDNS e depois Ctrl+C no terminal Django**. Se o Windows mostrar um aviso de firewall, permita o acesso.

9. Instalar o APK no smartphone Android

O APK de instalação será fornecido na **raiz do repositório**, com o nome definido para a entrega:

D:\Tcc\varificador_precos.apk

Após baixar o projeto pelo GitHub, localize esse arquivo e copie-o para o celular por cabo USB ou outro meio. No Android, abra o arquivo, autorize a instalação por essa origem se solicitado e conclua a instalação. **Não é necessário Android Studio para instalar o APK pronto.**

10. Executar o teste completo pelo celular

Execute o arquivo “**iniciar-servidor.bat”** escolha o modo **2 — REDE**.

Escolha a interface/IP que compartilha a rede do Android **Ex: 1 - Wi-Fi: 192.168.100.10**

Conecte o celular à mesma rede local do computador.

Abra o aplicativo **Verificador de Preços** e conceda a permissão de câmera quando solicitada.

O aplicativo localiza o Django pelo anúncio mDNS, verifica a API e configura a conexão. Se houver mais de um servidor compatível, permite selecionar um. Se a descoberta falhar, utilize a **configuração manual**, informando o IP atual do computador e a porta **8000**.

Use a camera para ler um dos códigos de barras de teste. Estão na pasta **“Tcc\Docs\Codigos_Testes”** ou digite um dos códigos: **7896277200136** ou **12345670**.

O aplicativo deverá mostrar nome, descrição e preço; A imagem será exibida quando houver um arquivo WebP correspondente e acessível. Utilize **Nova leitura** para repetir a consulta.

Para confirmar a API no navegador do computador, acesse http://127.0.0.1:8000/admin/ (painel administrativo) ou http://127.0.0.1:8000/api/docs/ (Swagger). Para consulta direta, abra **http://IP_DO_SERVIDOR:8000/api/produtos/CODIGO/** no endereço do servidor.

11. Solução rápida de problemas

Situação	O que conferir
.bat não inicia	A pasta .venv foi criada e requirements.txt foi instalado?
Django informa falha MySQL	Serviço ativo, porta 3306, usuário/senha e DB_NAME no .env.
DisallowedHost	Incluir o IP atual em DJANGO_ALLOWED_HOSTS e reiniciar Django.
MySQL não reconhece comando de importação	Use o Prompt de Comando com mysql no PATH ou abra produtos_demonstracao.sql no MySQL Workbench.
Nenhum produto aparece	Execute SELECT COUNT(*) FROM verificador_precos.produtos; (esperado: 5). Confirme que migrate terminou e que o SQL foi importado.
Erro de código de barras duplicado	Importe produtos_demonstracao.sql apenas uma vez, após migrate, com a tabela produtos vazia.
Android não encontra o servidor	Mesmo Wi-Fi, sem isolamento, firewall liberando TCP 8000 e multicast UDP 5353; tentar conexão manual.
Produto aparece sem foto	Verificar o arquivo .webp, a pasta media/img_produtos/ e IMAGENS_BASE_URL no **.env**.
O IP mudou	Atualizar DJANGO_ALLOWED_HOSTS e IMAGENS_BASE_URL, reiniciar o sistema.

12. Resultado esperado e observações para entrega

A instalação estará concluída quando o Django iniciar, o banco tiver os produtos cadastrados, o aplicativo encontrar o servidor e uma consulta por câmera ou digitação apresentar os dados do produto. O carregamento da foto depende do WebP e da URL correspondente.

Este projeto é um **protótipo acadêmico**. O servidor runserver, o uso de DJANGO_DEBUG=True e o HTTP local são destinados à demonstração em rede confiável, não à implantação pública.